Customize a field

Subclass an existing field component

Let's take an example where we want to extends the `BooleanField` to create a boolean field displaying "Late!" in red whenever the checkbox is checked.

1. Create a new widget component extending the desired field component.

late_order_boolean_field.js

```
import { registry } from "@web/core/registry";
import { BooleanField } from "@web/views/fields/boolean/boo
import { Component, xml } from "@odoo/owl";

class LateOrderBooleanField extends BooleanField {
   static template = "my_module.LateOrderBooleanField";
}
```

2. Create the field template.

   The component uses a new template with the name `my_module.LateOrderBooleanField`. Create it by inheriting the current template of the `BooleanField`.

late_order_boolean_field.xml

```
<?xml version="1.0" encoding="UTF-8" ?>
<templates xml:space="preserve">
    <t t-name="my_module.LateOrderBooleanField" t-inherit="
        <xpath expr="//CheckBox" position="after">
              <span t-if="props.value" class="text-danger">
        </xpath>
    </t>
</templates>
```

3. Register the component to the fields registry.

4. Add the widget in the view arch as an attribute of the field.

```
<field name="somefield" widget="late_boolean"/>
```

Create a new field component

Assume that we want to create a field that displays a simple text in red.

1. Create a new Owl component representing our new field

my_text_field.js

```
import { standardFieldProps } from "@web/views/fields/standa
import { Component, xml } from "@odoo/owl";
import { registry } from "@web/core/registry";

export class MyTextField extends Component {
   static template = xml`
      <input t-att-id="props.id" class="text-danger" t-att-v
   `;
   static props = { ...standardFieldProps };
   static supportedTypes = ["char"];

   /**
   * @param {boolean} newValue
   */
   onChange(newValue) {
      this.props.update(newValue);
   }
}
```

The imported `standardFieldProps` contains the standard props passed by the `View` such as the `update` function to update the value, the `type` of the field in the model, the `readonly` boolean, and others.

2. In the same file, register the component to the fields registry.

This maps the widget name in the arch to its actual component.

3. Add the widget in the view arch as an attribute of the field.

```
<field name="somefield" widget="my_text_field"/>
```

Get Help

Contact Support | Ask the Odoo Community